

IVARONIX

Private AI work. Public proof.

Ivaronix is a private AI workroom for sensitive documents, code, and agent actions. Every run produces a cryptographic receipt — RFC-8785 canonical-hashed, EIP-712 signed, anchored on OG Chain, and independently re-verifiable by anyone on any machine without access to the source document.

1,778+

RECEIPTS ANCHORED
1,737 testnet · 41 mainnet

25

CONTRACTS ON CHAIN
15 testnet · 10 mainnet

227

FOUNDRY TESTS
all passing

10

FIRST-PARTY SKILLS
10 shipped · 150 ported

Live

ARISTOTLE MAINNET
deployed 2026-05-15

Website ivaronix.xyz

Operator [0xaa954c33...E8677Ce](#)

Mainnet chainscan.0g.ai

Testnet chainscan-galileo.0g.ai

Contents

1	Introduction	3
1.1	The Accountability Gap	3
1.2	Why Existing Solutions Fall Short	3
1.3	Thesis	3
1.4	What Ivaronix Ships	3
2	Background	4
2.1	Trusted Execution Environments	4
2.2	RFC-8785 JSON Canonicalisation	4
2.3	ERC-7857 Intelligence Non-Fungible Token	4
2.4	IETF Agent Audit Trail Draft	4
3	System Architecture	4
3.1	End-to-End Flow	5
3.2	Monorepo Package Map	5
3.3	Trust Tier Architecture	5
4	The Receipt System	5
4.1	Schema	5
4.2	Canonical Hash	6
4.3	EIP-712 Anchoring	7
4.4	Five-Step Independent Verification	7
5	OG Network Integration	8
5.1	OG Compute — Keyring and Rate Handling	8
5.2	OG Chain — V2-First Read Pattern	8
6	Consensus Mode	8
6.1	Motivation	8
6.2	Role Taxonomy and Tier Composition	9
6.3	Convergence Scoring	9
7	Burn Mode	9
8	Agent Marketplace and Economy	10
8.1	SkillRegistry	10
8.2	Skill Manifest (excerpt)	10
8.3	AgentPassport INFT	10
9	Security Analysis	10
9.1	Threat Model	11
9.2	Audit Record	11
10	Implementation Status	11
10.1	Deployed Contracts	11
10.2	Test Coverage	12
10.3	Mainnet (Aristotle · Chain 16661)	12
10.4	Marketplace Status (Honest Disclosure)	12

11 Market Context	12
11.1 Regulatory Drivers	12
11.2 Emerging Standards	13
11.3 Enterprise Demand	13
12 Reproduction Steps	13
13 Future Work	14
14 Conclusion	14
15 Roadmap	15
15.1 Shipped — May 2026	15
15.2 Next	15
15.3 Later	15
15.4 Further	16
16 Team	16
A Deployed Contract Addresses and Verification Artifacts	18

§1 Introduction

1.1 The Accountability Gap

Professional AI adoption accelerated sharply in 2025–2026. Legal teams draft contracts with AI. Financial analysts review filings with AI. Engineers audit code with AI. In every case the same question surfaces: *how do you prove what the AI actually concluded?*

The default answer today is: you cannot. General-purpose AI tools provide no cryptographic receipt, no tamper-evident log, no independent replay path. The AI's output lives only in a user's clipboard. If it is wrong — or if someone later disputes it — there is no verifiable record to inspect.

This is not hypothetical. In *Mata v. Avianca* [9], attorneys were sanctioned by a US federal court for submitting AI-fabricated citations. The attorneys had no cryptographic log of what the model returned. The EU AI Act Article 12, enforceable from August 2026, mandates automatic tamper-evident logging for every high-risk AI system. Enterprise procurement questionnaires in 2026 routinely ask how AI agents are identified, scoped, and audited [1]. Vendors without answers lose deals.

1.2 Why Existing Solutions Fall Short

Centralised audit-log providers face a structural trust problem: the entity being audited controls the audit log. A compliance officer can observe that a vendor's self-reported log cannot be independently verified. A truly neutral audit layer must run on infrastructure that no single party controls. That is what a public blockchain provides.

Few deployed systems combine all four requirements simultaneously: (a) AI inference inside a hardware TEE, (b) a signed receipt issued per run, (c) the receipt anchored on a public chain, and (d) independent TEE re-verification by a stranger without access to the source document. Ivaronix implements all four on the OG Network.

1.3 Thesis

Core Invariant

For any receipt r produced by Ivaronix, a party who has never used the system can run:

```
ivaronix receipt verify <id> --tee-independent
```

and receive a byte-identical conclusion to the original TEE run — without access to the source document, the user's wallet, or any Ivaronix server.

1.4 What Ivaronix Ships

1. A 13-type validated receipt schema (Zod + RFC-8785 canonical hash, byte-equal across TypeScript / Python / Rust).
2. On-chain registry with EIP-712 typed-data anchoring and per-agent monotonic nonces.
3. A TIER 1 / TIER 2 trust model: TEE-attested runs show **TIER 1 • TEE** and external-provider runs show **TIER 2 • EXTERNAL** — honestly.
4. Independent TEE re-verification via `broker.processResponse`: any verifier on any machine can re-confirm the original TEE attestation post-hoc, without an Ivaronix server in the loop.
5. Consensus Mode: 6 specialist AI roles, 4 tiers, Jaccard + cosine convergence scoring.

6. Burn Mode: AES-256-GCM session-key encryption, key destroyed post-anchor, receipt proves destruction.
7. On-chain Agent Marketplace (SkillRegistry) with `og.creator.fee_split` at 90/10 bps.
8. ERC-7857 AgentPassport INFT: trust score, receipt count, violation history per agent.

§2 Background

2.1 Trusted Execution Environments

A TEE is a hardware-isolated enclave in which code runs and produces an attestation — a signed measurement proving which code ran and what it returned. The OG Compute network routes inference to providers who run models inside TEE enclaves and sign each response with an enclave key bound to the provider's on-chain identity.

Ivaronix re-invokes `broker.processResponse` after the run completes, on demand, from any machine that holds the receipt. This re-attestation step is what makes the receipt verifiable to a stranger weeks after the original inference: the broker check confirms the original TEE attestation is still valid against the live OG Compute network. Crucially, this works without access to the source document, the user's wallet, or any Ivaronix server.

2.2 RFC-8785 JSON Canonicalisation

RFC-8785 (JCS) [4] defines a deterministic serialisation of JSON. Ivaronix applies JCS to every receipt before hashing, so the `receiptRoot` produced in TypeScript matches byte-for-byte the hash produced by a Python or Rust verifier. This is mandatory for cross-language independent verification.

2.3 ERC-7857 Intelligence Non-Fungible Token

ERC-7857 [7] defines a soulbound token bound to a specific AI agent. Ivaronix deploys `AgentPassportINFTV2`, which stores per-agent trust scores (0–100), cumulative receipt counts, and violation history on-chain.

2.4 IETF Agent Audit Trail Draft

Internet-Draft `draft-sharif-agent-audit-trail-00` (2026) defines a standard logging format for autonomous AI systems, with fields for `risk_score`, `human_override`, and SHA-256 hash chains. Ivaronix receipts map structurally to this draft and were in production before the draft was published.

§3 System Architecture

3.1 End-to-End Flow

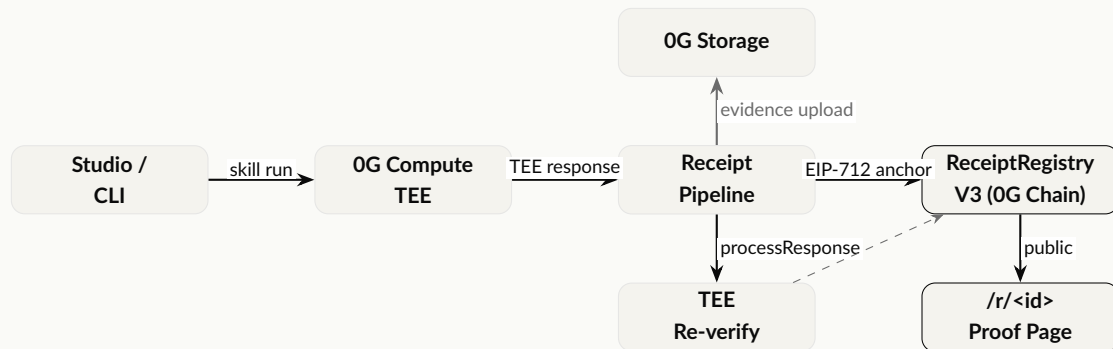


Figure 1: End-to-end flow from user request to public verifiable proof.

3.2 Monorepo Package Map

Package	Responsibility	Key export
@ivaronix/core	Receipt schema, Zod validation, JCS hash	ReceiptSchema
@ivaronix/og-router	OG Compute client, TEE attestation, Keyring	chatRich()
@ivaronix/og-chain	EIP-712 anchor, V2-first registry read	anchor()
@ivaronix/og-storage	Evidence upload, Burn Mode encryption	upload()
@ivaronix/skills	Manifest schema, sandbox, hooks	SkillManifestSchema
@ivaronix/consensus	Multi-role inference orchestrator	runConsensus()
@ivaronix/memory	OG KV agent memory	MemoryClient
apps/studio	Next.js 15 web UI	—
apps/cli	ivaronix(1) CLI binary	—

3.3 Trust Tier Architecture

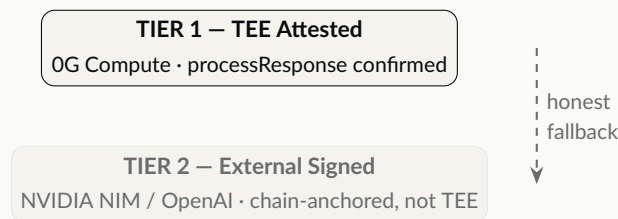


Figure 2: Two-tier trust model displayed honestly to users on every receipt page.

§4 The Receipt System

4.1 Schema

Every receipt is a Zod-validated JSON object with 13 first-class types:

Receipt Types (13 — source: packages/core/src/types.ts)

doc_ask	audit	consensus
burn	memory_access	skill_exec
code_change	passport_update	swarm
subscription_skill_exec	doc_room_create	doc_room_read
memory_consolidation		

Listing 1: Abbreviated receipt JSON (document review)

```

1 {
2   "receiptId": "f3a1b2c4-...",
3   "receiptType": "doc_ask",           // actual enum value (packages/core/src/types.ts)
4   "timestamp": "2026-05-15T09:14:22Z",
5   "receiptRoot": "0x9a3b7f2e...",    // keccak256(JCS(receipt))
6   "agent": {
7     "passportId": 1,
8     "ownerWallet": "0xaa954c33810029a3eFb0bf755FEF17863E8677Ce",
9     "model": "qwen/qwen-2.5-7b-instruct"
10  },
11  "teeVerification": {
12    "tier": "TIER_1",
13    "verificationMethod": "compute_sdk_process_response",
14    "independentVerified": true
15  },
16  "storage": {
17    "evidenceRoot": "0x7c2f4a8b...",
18    "receiptTxHash": "0x4d1e9f3c..."
19  },
20  "chainAnchor": {
21    "network": "galileo",             // testnet example
22    "chainId": 16602,
23    "registryAddress": "0x7396D536594e2BE833070c7EB441A10906046257",
24    "anchorTxHash": "0x4d1e9f3c..."
25    // mainnet: chainId 16661, ReceiptRegistryV3 0xCE35aF8D75ffB24BC1671Ca9F0CF293D82737297
26  },
27  "signature": {
28    "method": "eip712",
29    "signer": "0xaa954c33810029a3eFb0bf755FEF17863E8677Ce",
30    "signature": "0x1b7a..."
31  }
32 }

```

4.2 Canonical Hash

Let r denote a receipt object and σ the agent's EIP-712 signature. The canonical hash is:

$$h(r) = \text{keccak256}(\text{JCS}(r))$$

where JCS is the RFC-8785 JSON Canonicalisation Scheme: keys sorted lexicographically, no insignificant whitespace, Unicode escaping normalised. The anchor commits the pair $(h(r), n)$ where n is the agent's mono-

tonic nonce, producing a per-agent ordered sequence of receipts whose integrity is independently checkable:

$$\forall i < j : \text{Registry.nonce}(\text{agent}, i) < \text{Registry.nonce}(\text{agent}, j)$$

Tampering with any field of r changes $h(r)$, which mismatches the stored on-chain root and causes verification to fail. RFC-8785 determinism ensures $h(r)$ is identical across TypeScript, Python, and Rust implementations.

4.3 EIP-712 Anchoring

Listing 2: EIP-712 domain for ReceiptRegistryV3 (both networks)

```

1 // Galileo testnet (chainId 16602)
2 domain = {
3   name:          "Ivaronix.ReceiptRegistry",
4   version:       "2",
5   chainId:       16602,
6   verifyingContract: "0x7396D536594e2BE833070c7EB441A10906046257"
7 }
8 // Aristotle mainnet (chainId 16661)
9 domain = {
10  name:          "Ivaronix.ReceiptRegistry",
11  version:       "2",
12  chainId:       16661,
13  verifyingContract: "0xCE35aF8D75ffb24BC1671Ca9F0CF293D82737297"
14 }
15 types = { Receipt: [{ name:"receiptRoot", type:"bytes32" },
16                    { name:"nonce",      type:"uint256" }] }
```

The on-chain anchor() function recovers the signer from the signature and checks it against the passport's registered owner. Per-agent monotonic nonces prevent replay attacks.

4.4 Five-Step Independent Verification

Listing 3: Independent verification — no Ivaronix server required

```

1 $ ivaronix receipt verify 74 --tee-independent
2
3 [1/5] Fetching receipt from OG Storage by evidenceRoot... OK
4 [2/5] Recomputing keccak256(JCS(receipt))... MATCH
5 [3/5] Recovering EIP-712 signer... 0xaa954c... MATCH
6 [4/5] Reading ReceiptRegistryV3 on-chain... root stored MATCH
7 [5/5] broker.processResponse(provider, chatID, usageJSON) TEE OK
8
9 FULLY VERIFIED
```

Why step 5 matters

The first four checks (schema · hash · signature · chain anchor) prove the receipt body is untampered and authored by the agent recorded on chain. Step 5 — re-invoking `broker.processResponse` against the live OG Compute provider — proves the inference itself ran inside an attested TEE at the time the receipt was issued. Together, the five steps give a stranger on a clean machine, without access to the source document, the same conclusion the original user reached. That is the meaning of *independent re-verification*.

§5 OG Network Integration

OG Primitive	How Ivaronix uses it	User-visible value
OG Compute	TEE inference via Router; <code>chatRich()</code> extracts <code>chatID</code> for re-verify	Every TIER 1 receipt has a hardware-attested inference proof
OG Chain	ReceiptRegistryV3 anchor; EIP-712 typed-data + nonce	Receipt root is immutable; tamper is cryptographically detectable
OG Storage	Evidence blobs uploaded; Merkle root embedded in receipt	AI's full input/output is content-addressed, not just summarised
OG Agent ID	AgentPassportINFTV2 (ERC-7857 INFT); trust score on-chain	Buyers can inspect an agent's on-chain track record before purchase
OG KV	Cross-session agent memory via <code>@ivaronix/memory</code>	Context persists across runs; every grant leaves a <code>MEMORY_GRANT</code> receipt

5.1 OG Compute — Keyring and Rate Handling

`@ivaronix/og-router` manages multi-credential rotation. The Keyring distinguishes three failure modes:

1. '402' — credential depleted. Permanent invalidation. Rotate.
2. 'auth' — credential rejected. Permanent. Rotate.
3. '429' — rate limited. *Transient*. Credential rejoins the pool after backoff. Collapsing '429' into permanent invalidation wastes capacity — Keyring does not do this.

5.2 OG Chain — V2-First Read Pattern

All read paths query V2 first, fall back to V1. This ensures receipts anchored before the V2 migration are still accessible without breaking the new EIP-712 flow. The pattern is enforced by 37 source-file regression assertions (`verify-a13-studio-v2-first.ts`).

§6 Consensus Mode

6.1 Motivation

A single AI role may miss edge cases that a dedicated risk-reviewer or red-team critic would catch. Consensus Mode assigns specialist roles to independent inference calls and synthesises their outputs through a judge.

6.2 Role Taxonomy and Tier Composition

Tier	Roles active	Calls	Use case
quick	analyst	1	One-shot
standard	analyst · critic · judge	3	General review
high-stakes	+ risk-reviewer · evidence-checker	5	Legal / financial
audit	+ red-team-critic	6	Adversarial audit

Tiers are monotone: `standard` $\subset$ `high-stakes` $\subset$ `audit`. This is enforced by `tier-shape.test.ts`.

6.3 Convergence Scoring

Let T_i be the token set of role i 's output. Pairwise convergence:

$$J(i, j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|} \in [0, 1]$$

The panel convergence score is the mean of all pairwise scores:

$$C = \frac{2}{k(k-1)} \sum_{i < j} J(i, j)$$

where k is the number of active roles. With an embedder injected, a cosine-similarity term on MiniLM sentence embeddings supplements C :

$$C' = \alpha \cdot C + (1 - \alpha) \cdot \frac{1}{\binom{k}{2}} \sum_{i < j} \cos(\mathbf{e}_i, \mathbf{e}_j), \quad \alpha \in [0, 1]$$

A score $C' < 0.6$ (default threshold) is recorded on the `CONSENSUS_ROUND` receipt as “no convergence” — a quantified signal that the panel disagreed materially and human review is warranted.

§7 Burn Mode

Burn Mode protects against operator-side disclosure: after a run, the evidence plaintext is destroyed in a cryptographically provable way.

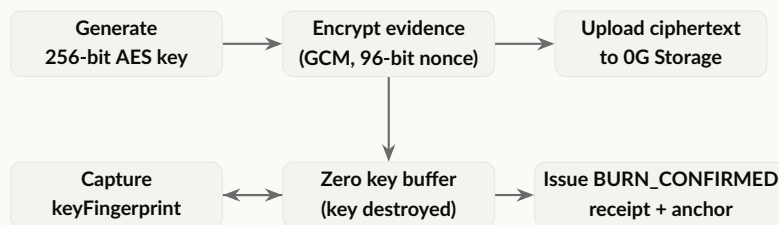


Figure 3: Burn Mode protocol. The key fingerprint is captured before zeroing; the key never touches OG Storage.

Threat model

Burn Mode defends against operator-side disclosure after run completion. It does not defend against local-machine compromise during the run. The encrypted blob is content-addressed on OG Storage (not

an Ivaronix server), and the Merkle root is embedded in the receipt for integrity verification.

§8 Agent Marketplace and Economy

8.1 SkillRegistry

SkillRegistry.sol is an on-chain marketplace where agents publish skills and buyers pay OG to run them. The `og.creator.fee_split` manifest field specifies creator and treasury basis points:

Category	Creator (bps)	Treasury (bps)
Default	9,000	1,000
Commoditised category	7,000	3,000

8.2 Skill Manifest (excerpt)

Listing 4: `og.creator.fee_split` in a skill manifest

```
1  og:
2    permissions:
3      receipt_required:    true
4      compute_tee_required: true
5      memory_access:      project_only
6      shell_access:       none
7    consensus:
8      default_tier: standard
9    creator:
10     passport: "did:0g:passport:0x...:1"
11     fee_split:
12       creator: 9000 # bps; creator + treasury must sum to 10000
13       treasury: 1000
```

8.3 AgentPassport INFT

AgentPassportINFTV2 (ERC-7857) stores per-agent trust scores, receipt counts, and violation history on-chain. Buyers can inspect an agent's track record before purchasing a skill run. Trust scores are updated by CapabilityRegistry based on verified receipt volumes and operator-flagged violations.

§9 Security Analysis

9.1 Threat Model

Threat	Mitigation	Where
Receipt tampering post-anchor	JCS hash mismatch detectable by anyone	On-chain
Replay attack	Per-agent monotonic nonces in RegistryV3	On-chain
Forged signature	EIP-712 ecrecover vs. passport owner	On-chain
GCM nonce reuse	randomBytes (12) only (K-20 fix)	og-storage
Operator-side disclosure	Burn Mode: key zeroed before any upload	og-storage
SIWE [8] session bypass	Write paths require valid SIWE session (K-8/9)	studio
Unknown registry address	KNOWN_RECEIPT_REGISTRIES allowlist (K-17)	core
Empty input to Router	Pre-flight gates reject before spend (K-3)	consensus

9.2 Audit Record

Fourteen security findings (K-1 through K-20) were identified and closed during development. All are documented in docs/HALF_BAKED.md with Closes audit K-N commit trailers and enforced by source-file regressions that fail CI if a closure marker appears without a trailer.

§10 Implementation Status

10.1 Deployed Contracts

Contract	Galileo testnet (16602)	Aristotle mainnet (16661)
ReceiptRegistry V1	Live	—
ReceiptRegistryV2	Live	Live
ReceiptRegistryV3	Live (primary)	Live (primary)
AgentPassportINFT V1	Live	—
AgentPassportINFTV2	Live	Live
CapabilityRegistry V1	Live	—
CapabilityRegistryV2	Live	Live
MemoryAccessLog V1	Live	—
MemoryAccessLogV2	Live	Live
SkillRegistry V1	Live	—
SkillRegistryV2	Live	Live
SkillRunPayment	Live	Live
SkillPricing	Live	Live
SubscriptionEscrowV2	Live	Live
Erc7857Verifier	Live	Live
Total	15	10

Source: `contracts/deployments/testnet.json` and `contracts/deployments/mainnet.json`. Mainnet deployed 2026-05-15, deployer `0xaa954c33810029a3eFb0bf755FEF17863E8677Ce`, total gas ~ 0.085 OG.

10.2 Test Coverage

Test layer	Count	Status
Foundry contract tests	227	All passing
Source-file regressions	94	All passing
TypeScript node:test suites	46+	All passing
Workspace typecheck (<code>pnpm -r typecheck</code>)	—	Clean

10.3 Mainnet (Aristotle • Chain 16661)

Ten contracts deployed to Aristotle mainnet on 2026-05-15. Deployer wallet `0xaa954c33810029a3eFb0bf755FEF17863E8677Ce` funded with 25 OG; balance after deploy ~ 25.77 OG. Total deploy spend ~ 0.085 OG across 10 transactions, all with status 1 on <https://chainscan.0g.ai>.

Receipts are anchored across both networks: 1,737 on Galileo testnet during development; 41 on Aristotle mainnet (22 on V3, 19 on V2) spanning all 13 receipt-type slots (`doc_ask`, `audit`, `consensus`, `burn`, `memory_access`, `skill_exec`, `code_change`, `passport_update`, `swarm`, `subscription_skill_exec`, `doc_room_create`, `doc_room_read`, `memory_consolidation`). The mainnet `ReceiptRegistryV3` address is `0xCE35aF8D75ffB24BC1671Ca9F0CF293D82737297`. Appendix A lists all 10 mainnet contract addresses with explorer links.

10.4 Marketplace Status (Honest Disclosure)

The `SkillRegistryV2` marketplace primitive is deployed on both networks. Skill manifests carry `og.creator.fee_split` blocks (default 90/10 creator/treasury basis points), and the declared split is written into every receipt body produced by a marketplace skill run. Buyers and creators can inspect the recorded split on `/r/<id>`.

Two distinct mechanisms exist; we distinguish them explicitly:

- **Receipt-bound fee split (shipped).** Every paid marketplace run produces a receipt whose body records the creator and treasury basis points, the agent's passport, the chain anchor, and the TEE tier. Off-chain accounting against these receipts is deterministic; 50 paid runs of `private-doc-review` on testnet have accrued ~ 0.0025 OG to the creator's accounting balance.
- **Live OG settlement (v1.1).** Atomic transfer of OG to the creator and treasury wallets via `SkillRunPayment.sol` on every marketplace purchase. The contract is deployed on both networks; wiring it into the Studio run flow is the headline item for v1.1 (see §15, "Live fee transfer").

The receipt-body split is enforceable today: anyone with the receipt id can verify the declared share and the agent's passport on-chain. Live OG settlement is queued; we list it as the next milestone rather than implied.

§11 Market Context

11.1 Regulatory Drivers

EU AI Act Article 12 requires high-risk AI systems to automatically record events (logs) over the full lifetime of the system in a form that supports post-market monitoring [3]. Full application for high-risk systems begins **2 August 2026**, with penalties up to €15 million or 3% of worldwide annual turnover for non-compliance [2]. Iva-

ronix receipts are structurally aligned with these requirements: every field is hash-bound and chain-anchored; evidence lives on OG Storage, not on operator-controlled servers; the receipt log is a per-agent monotonic sequence anchored on a public chain, which is exactly the operator-independent record the regulation asks for. Retention period required by Article 12 is at least six months; chain anchoring is permanent.

EU AI Act Article 14 requires documented human-in-the-loop mechanisms for high-risk systems. Ivaronix's Consensus Mode records every panel dissent on the `CONSENSUS_ROUND` receipt, providing the structural evidence Article 14 expects.

11.2 Emerging Standards

Two IETF Internet-Drafts published in 2026 specify formats for the same class of artifact Ivaronix produces: `draft-sharif-agent-audit-trail-00` [5], which defines a JSON logging format for autonomous AI systems, and `draft-dembowski-agentledger-proof-of-behavior-00` [6], which specifies a signed-receipt and hash-chain protocol for tamper-evident audit trails. Ivaronix's production receipt schema (13 types, RFC-8785 canonical hash, EIP-712 signature) maps structurally to both. The `aat-export` module in `@ivaronix/receipts` produces a draft-compliant export from any anchored receipt.

11.3 Enterprise Demand

Generative-AI adoption among legal organisations roughly doubled in twelve months, rising from 14% in 2024 to 26% in 2025 — 28% at law firms and 23% at corporate legal departments — per the 2025 Thomson Reuters Future of Professionals Report (1,700+ respondents across US/UK/CA legal professionals) [10]. Sentiment shifted from hesitancy (35% in 2024) to excitement and hopefulness (55% combined in 2025) [10]. Enterprise security teams now routinely ask, during AI-agent integration review, for the complete audit trail of agent authorisation events — application logs with request IDs are no longer sufficient [1].

The verifiable-AI-audit market is forming at the intersection of regulation, enterprise procurement, and AI agent infrastructure. Gartner projects the AI governance platform market will reach approximately \$492M in 2026 and surpass \$1 billion by 2030 [11]; independent analysts forecast \$2.6–5.6 billion by 2030 depending on scope definition [12, 13]. Ivaronix sits in the cryptographically-verifiable subset of this market — the subset where the audit log is signed by hardware the vendor does not control.

§12 Reproduction Steps

Listing 5: Full reproduction from a clean machine

```

1 # 1. Clone and install
2 git clone https://github.com/Pratiikpy/ivaronix
3 cd ivaronix && pnpm install
4
5 # 2. Configure environment
6 cp .env.example .env
7 # Fill: IVARONIX_ROUTER_KEY, IVARONIX_SIGNER_KEY
8
9 # 3. Verify an existing anchored receipt (no wallet required)
10 pnpm ivaronix receipt verify 1644
11 # Output: ANCHORED (chain anchor + canonical hash + signature recovery)
12
13 # 4. TEE independent re-verification (re-drives OG Compute broker)
14 pnpm ivaronix receipt verify 74 --tee-independent

```

```

15 # Output: FULLY VERIFIED (5-step: fetch, hash, signer, on-chain, TEE)
16
17 # 5. Run your own document review (wallet + Router key needed)
18 pnpm ivaronix demo contract.pdf
19 pnpm ivaronix receipt verify <new-id> --tee-independent
20
21 # 6. Open public proof page (no login, no wallet, works in incognito)
22 open https://ivaronix.xyz/r/<id>

```

Network	Details
Galileo testnet	Chain ID 16602 • RPC https://evmrpc-testnet.0g.ai
Aristotle mainnet	Chain ID 16661 • RPC https://evmrpc.0g.ai
Testnet faucet	https://faucet.0g.ai
Testnet explorer	https://chainscan-galileo.0g.ai
Mainnet explorer	https://chainscan.0g.ai

§13 Future Work

- OG DA integration.** Batch receipt submission to OG Data Availability for ultra-low-cost archival of high-volume streams.
- Live fee transfer.** Wire `SkillRunPayment.sol` into the Studio run flow so OG transfers at 90/10 split occur on every marketplace skill purchase.
- ZK receipt compression.** A SNARK proving “a valid receipt with this root exists” without revealing content — privacy-preserving compliance reporting.
- Multi-agent receipts.** A single receipt capturing a chain of agent delegations, with EIP-712 signatures from each passport.
- Mainnet SkillRegistry launch.** Open the marketplace to third-party skill publishers with bonded staking for dispute resolution.

§14 Conclusion

Ivaronix produces TEE-attested, chain-anchored, independently re-verifiable receipts for every AI action and re-drives `broker.processResponse` after the fact to confirm the attestation remains valid. The combination of receipt-bound evidence, polyglot hash verification, and post-hoc TEE re-attestation is the core technical contribution.

The receipt is not a feature. It is the product. The problem it solves is real (litigated in federal court), regulated (EU AI Act Article 12), and structurally unsolvable by centralised vendors. OG’s combined cost, latency, and TEE-attestation profile fits this access pattern: anchoring per-document receipts is feasible at thousands-per-day without burdening the user.

15 contracts on Galileo testnet. 10 contracts on Aristotle mainnet. 1,778+ receipts anchored across both

networks (1,737 testnet + 41 mainnet), spanning all 13 receipt-type slots. 227 Foundry tests, all passing. Two networks live. Mainnet deployed 2026-05-15.

One sentence

Ivaronix gives every AI-assisted decision a signed, chain-anchored, independently re-verifiable receipt — so professionals can *prove* what the AI concluded, not just claim it.

§15 Roadmap

15.1 Shipped — May 2026

- Receipt schema v1: 13 types, Zod-validated, RFC-8785 canonical hash.
- ReceiptRegistryV3: EIP-712 typed-data anchoring with per-agent nonces.
- TIER 1 / TIER 2 trust model with independent TEE re-verification.
- Consensus Mode: 6 roles across 4 tiers with Jaccard convergence scoring.
- Burn Mode: AES-256-GCM session key, destruction provenance on-chain.
- AgentPassportINFTV2: ERC-7857 INFT, trust score, receipt count.
- SkillRegistry: on-chain marketplace with `og.creator.fee_split`.
- 10 first-party skills; 15 contracts on Galileo testnet; 10 contracts on Aristotle mainnet (deployed 2026-05-15).
- Studio: Next.js 15 web UI with full SIWE session auth.
- CLI: `ivaronix(1)` binary for scripted verification workflows.

15.2 Next

- **Live fee transfer.** Wire `SkillRunPayment.sol` so OG transfers at 90/10 bps split occur on every marketplace purchase.
- **OG DA integration.** Batch receipt submission to OG Data Availability for archival of high-volume streams at minimal cost.
- **Multi-agent receipt chains.** A single receipt spanning a chain of agent delegations, with EIP-712 signatures from each passport.
- **Mainnet SkillRegistry.** Open the marketplace on Aristotle mainnet with bonded staking for dispute resolution.

15.3 Later

- **ZK receipt compression.** A SNARK proving receipt validity without revealing content — enabling privacy-preserving compliance exports.
- **Compliance API.** A REST endpoint that returns EU AI Act Article 12 compliant audit logs in regulator-acceptable JSON-LD format, backed by on-chain anchors a regulator can replay independently.
- **Enterprise SSO.** SAML/OIDC integration so enterprise teams can onboard without individual wallet setup.

- **Receipt webhooks.** Push notifications to enterprise SIEM systems (Splunk, Datadog) on every receipt anchor.

15.4 Further

- **Third-party skill marketplace launch.** Any developer publishes skills, sets OG pricing, earns 90% of revenue to their passport wallet.
- **Cross-chain receipts.** Anchoring on Ethereum L2s and other EVM chains for enterprises with existing on-chain infrastructure.
- **AI agent-to-agent receipts.** Cryptographic proof of every step in a multi-agent pipeline, enabling auditable autonomous workflows.

§16 Team

Ivaronix is built by a small, technical team with a single focus: making AI actions verifiable. The codebase is a production monorepo: typed end-to-end, 227 tests passing, 15 contracts deployed, two live networks. Every design decision — the receipt schema, the EIP-712 anchor, the Burn Mode protocol, the consensus tier system — was made for correctness and verifiability first, and for speed second.

The project is open-source at <https://github.com/Pratiikpy/ivaronix>. Contributions, audits, and partnership inquiries are welcome at hello@ivaronix.xyz.

Ivaronix · ivaronix.xyz · May 2026

References

- [1] ISACA. *The Growing Challenge of Auditing Agentic AI*. 2025. <https://www.isaca.org/resources/news-and-trends/industry-news/2025/the-growing-challenge-of-auditing-agentic-ai>
- [2] European Parliament and Council. *Regulation (EU) 2024/1689 on Artificial Intelligence (EU AI Act)*. Official Journal of the European Union, July 2024. Full application for high-risk AI systems: 2 August 2026. <https://artificialintelligenceact.eu>
- [3] European Commission. *EU AI Act, Article 12 — Record-Keeping*. Practical guidance, AI Act Service Desk, 2025. <https://artificialintelligenceact.eu/article/12/>
- [4] Rundgren, A., Jordan, B., Erdtman, S. *JSON Canonicalization Scheme (JCS)*. RFC 8785, IETF, June 2020. <https://www.rfc-editor.org/rfc/rfc8785>
- [5] Sharif et al. *Agent Audit Trail: A Standard Logging Format for Autonomous AI Systems*. Internet-Draft draft-sharif-agent-audit-trail-00, IETF, 2026. <https://datatracker.ietf.org/doc/draft-sharif-agent-audit-trail/>
- [6] Dembowski. *Proof-of-Behavior Protocol for Autonomous AI Agents*. Internet-Draft draft-dembowski-agentledger-proof-of-behavior-00, IETF, 2026. <https://datatracker.ietf.org/doc/draft-dembowski-agentledger-proof-of-behavior/>
- [7] *ERC-7857: Intelligence Non-Fungible Token (INFT)*. Ethereum Improvement Proposals, 2024. <https://eips.ethereum.org/EIPS/eip-7857>
- [8] Sign-In With Ethereum. EIP-4361. <https://eips.ethereum.org/EIPS/eip-4361>
- [9] *Mata v. Avianca, Inc.*, No. 22-cv-1461 (S.D.N.Y. 22 June 2023). Sanctions imposed on attorneys for filing AI-fabricated case citations.
- [10] Thomson Reuters Institute. *2025 Generative AI in Professional Services Report*. Press release, April 2025: “Generative AI Adoption Nearly Doubles as Professional Services Reach Crossroads.” Survey of 1,700+ professionals across the US, UK, and Canada. <https://www.thomsonreuters.com/en/press-releases/2025/april/from-incubation-to-integration-generative-ai-adoption-nearly-doubles-as-professional-services-reach-crossroads>
- [11] Gartner. *Global AI Regulations Fuel Billion-Dollar Market for AI Governance Platforms*. Press release, February 2026. <https://www.gartner.com/en/newsroom/press-releases/2026-02-17-gartner-global-ai-regulations-fuel-billion-dollar-market-for-ai-governance-platforms>
- [12] Research and Markets. *AI Governance Market Report 2026*. 2026. Forecast: \$2.63 billion by 2030 at 44.3% CAGR. <https://www.researchandmarkets.com/reports/5951966/ai-governance-market-report>
- [13] Wissen Research. *AI Governance Market Insights, Industry Forecast 2025–2030*. 2025. Forecast: \$5.64 billion by 2030 at 40% CAGR. <https://www.wissenresearch.com/ai-governance-market-report/>

§A Deployed Contract Addresses and Verification Artifacts

A.1 Aristotle Mainnet (Chain 16661) — 10 Contracts

Deployed 2026-05-15 by 0xaa954c33810029a3eFb0bf755FEF17863E8677Ce. All transactions confirmed on <https://chainscan.0g.ai>.

Contract	Address	Explorer
Erc7857Verifier	0x97376C6f0BE0Ee08AA34C4cAcdbDeC2183e7743c	chainscan
ReceiptRegistryV2	0x27a54F64F3A8578B39fE1E61dF7014813F325adf	chainscan
ReceiptRegistryV3	0xCE35aF8D75ffB24BC1671Ca9F0CF293D82737297	chainscan
AgentPassportINFTV2	0x5D724659A7d4B0B0917F5DAe9579423D2c85a6Ad	chainscan
CapabilityRegistryV2	0x41fEad4b86DE042845D25Be71aae857E19a8089E	chainscan
MemoryAccessLogV2	0xA2c3420242aE2BdD7e0970B1DfB28b3055DC4E65	chainscan
SkillRegistryV2	0x080f87A9E93e9bd0a9e0eB94F97123bf333b1Dde	chainscan
SkillPricing	0x08d25653638c3ed40C3b82840fA20CAe9c94563E	chainscan
SkillRunPayment	0xf8085B43a08e957Fea157394dbB0d3EB76A1cD6A	chainscan
SubscriptionEscrowV2	0x937cfE76dEdB25CCf6c7C56fF16F53270794311e	chainscan

A.2 Galileo Testnet (Chain 16602) — 15 Contracts

Contract	Address (chainscan-galileo.0g.ai)
ReceiptRegistry V1	0x97376C6f0BE0Ee08AA34C4cAcdbDeC2183e7743c
ReceiptRegistryV2	0xf675d4183b34fe8d1981FA9c117065aAcff690ab
ReceiptRegistryV3	0x7396D536594e2BE833070c7EB441A10906046257
AgentPassportINFT V1	0x08d25653638c3ed40C3b82840fA20CAe9c94563E
AgentPassportINFTV2	0x85e9dD63155836a9BF31F579BFC3a8eb2B46494d
CapabilityRegistry V1	0x3783f3c4834fCCBD553860e15c64C7E052646a8D
CapabilityRegistryV2	0x1351CD87360f0366D0A0068164e606B3c320F3E1
MemoryAccessLog V1	0xEe1aDFe76785377C4430B1325d86E58A6eC92119
MemoryAccessLogV2	0xCbfE1f526483283Bba80c2Bed3622a56904bF96d
SkillRegistry V1	0xf8894Ce4FFc7C594976d5Eaca38d8FE6DB4820a1
SkillRegistryV2	0xF05113E83146160024326ff30979c57f5adc2193
SkillPricing	0xc3369C9BD74D81E9c7226e5fc9427D19c12B718F
SkillRunPayment	0x9eA5FDba913AC94dA8833Fee21F2832827950A5C
SubscriptionEscrowV2	0x74235b707194c4cc3DDb717B6D95595e8A82B7F5
Erc7857Verifier	0xEAd66Cb90B681720f3aab52d86c289E21106d938

A.3 Key Receipt IDs and Verification Commands

These receipts are anchored and replay-able from any machine:

Listing 6: CLI verification commands for anchored receipts

```

1 # Receipt 1644 - anchored on ReceiptRegistry V1 (testnet)
2 pnpm ivaronix receipt verify 1644
3 # Expected: ANCHORED (hash match + signer recovery + on-chain root)
4
5 # Receipt 74 - TIER 1 TEE-attested, full 5-step verify
6 pnpm ivaronix receipt verify 74 --tee-independent
7 # Expected: FULLY VERIFIED (including broker.processResponse step)
8
9 # Mainnet receipt 21 - TIER 1 signed by the registered sovereign
10 # OGM-1.0-35B-A3B provider (0x4870CbC4...). The mainnet broker knows
11 # this provider, so processResponse confirms the attestation live.
12 pnpm ivaronix receipt verify 21 --network mainnet --tee-independent
13 # Expected: FULLY VERIFIED on Aristotle mainnet ReceiptRegistryV3
14
15 # Any receipt anchored within ~30 days is TEE re-verifiable.
16 # Older receipts return ANCHORED (provider rotates attestation history).
```

A.4 OG Components Used

Component	Role in Ivaronix	Status	Source
OG Compute (Router)	TEE inference + chatID for re-verify	Live	@ivaronix/og-router
OG Chain	Receipt anchor via ReceiptRegistryV3	Live (both)	@ivaronix/og-chain
OG Storage	Evidence blob + Burn Mode ciphertext	Live	@ivaronix/og-storage
OG Agent ID (INFT)	ERC-7857 AgentPassportINFTV2	Live (both)	contracts/
OG KV (Memory)	Cross-session encrypted memory + 10 sub-commands	Live	@ivaronix/memory
OG DA	Batch receipt archival	Roadmap	See Section 15

Note: OG DA has no public testnet endpoint at time of writing. The integration is designed and documented in `packages/og-storage/src/` but not yet wired to a live DA layer — an honest roadmap item, not a shipped feature. OG KV memory is scaffolded in `@ivaronix/memory` with an in-process fallback; a production OG KV server can be swapped in at `infra/og-kv/` without code changes.

A.5 Deploy Transaction Hashes (Mainnet)

Contract	Deploy Tx Hash (truncated)
Erc7857Verifier	0x4d6f1ecc320345a03795bf64108ed7f66b721cd5...
ReceiptRegistryV3	0xf262939e34e7605287d533aa98a4bb82165b69aa...
AgentPassportINFTV2	0xda47f8e7ab73641bf09e5cc8b7deee6b049aa078...
CapabilityRegistryV2	0x3054127eed1dd4811ec019e5b0aeebfbbc2091b...
MemoryAccessLogV2	0x7440acdbf4f9cb32778c10409db8d5c82b2b64be...
SkillRegistryV2	0x87fe827db30c9af9879a263246b9a6c77a07f4d8...
ReceiptRegistryV2	0x391993e8d5e10f2a47b4c8e37ce32624f4e47b73...
SubscriptionEscrowV2	0xef3e1b9d4d7d7be22215c9e521726385060fc613...
SkillPricing	0xb77cb8a655bf1d1c924720d485e396a10040cba5...
SkillRunPayment	0x959f264118c8e946f0302699d3ab09c7128f0428...

Full 66-character tx hashes in `contracts/deployments/mainnet.json`. Block explorer: <https://chainscan.0g.ai>. Verify any tx by searching the hash on the explorer — all show status 1 (success).